

Formalizing Filesystem Lifecycle Semantics: An
Object-Centric Process Mining Framework for
Autonomic Artifact Management

Sean Chatman

June 2026

Contents

1	Declaration of Authorship	3
2	Acknowledgements	4
3	Abstract	5
4	Glossary of Foundational Axioms, Laws, and Proofs	6
5	Chapter 1: Introduction	8
6	Chapter 2: Literature Review and State of the Art	9
7	Chapter 3: Petri Nets and Workflow Soundness	10
8	Chapter 4: Object-Centric Process Mining (OCPM) and OCEL	11
9	Chapter 5: Filesystem Ontologies and Event Abstraction	12
10	Chapter 6: The Universal Chatman Equation	13
11	Chapter 7: The Category Fab and Surface Relations	14
12	Chapter 8: The Gall Pipeline and Temporal Constraints	15
13	Chapter 9: Affine Typestate Guarantees	16
14	Chapter 10: Theoretical Limits and Rice's Theorem	17
15	Chapter 11: Cryptographic Receipts and BLAKE3 Commitments	18

16 Chapter 12: Time Machine and APFS Local Snapshots	19
17 Chapter 13: Autonomic Reinforcement Learning Ecosystems	20
18 Chapter 14: The Cfab Surface Graph and Pathfinding	21
19 Chapter 15: The Noun-Verb Substrate and CLI Architecture	22
20 Chapter 16: Wasm4pm Integration and OCPN Replay	23
21 Chapter 17: Empirical Evaluation and Baselines	24
22 Chapter 18: Cache Thrashing and Sliding Windows	25
23 Chapter 19: Privacy, Redaction, and Sanitization	26
24 Chapter 20: Appendix: Verification Index	27
25 References	28

Chapter 1

Declaration of Authorship

I, **Sean Chatman**, declare that this thesis titled, “*Formalizing Filesystem Lifecycle Semantics: An Object-Centric Process Mining Framework for Autonomic Artifact Management*” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree.
- Where any part of this thesis has previously been submitted for a degree or any other qualification, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given.

Signed: *Sean Chatman*

Date: June 2026

Chapter 2

Acknowledgements

I would like to express my deepest appreciation to my advisors and the broader Process Mining community. In particular, the foundational research of Prof. dr. ir. Wil M.P. van der Aalst on Object-Centric Process Mining provided the mathematical ontology required to map continuous filesystem entropy to discrete, executable process logic. I am also deeply indebted to the open-source Rust community for developing the affine typestate compiler guarantees that made the safety boundary possible.

Chapter 3

Abstract

The lifecycle of local software artifacts is currently managed through ad-hoc, heuristic-based scripts, leading to unbounded storage bloat, broken dependencies, and opaque data loss. We prove that deletion is a typed state transition admissible to the same OCEL (Object-Centric Event Log) ontology as creation, modification, and access. This thesis introduces a mathematically rigorous framework that applies Object-Centric Process Mining (OCPM) to local disk lifecycle management. By enforcing the deterministic “Gall Pipeline” via Rust’s `typestate` system and securing executions with unforgeable BLAKE3 receipt chains, we guarantee the safety of autonomic deletion policies. Through the integration of the `wasm4pm` engine, we formally evaluate alignment-based conformance checking over $N \geq 1,000$ traces. We demonstrate that process intelligence can transition filesystem management from static measurement to autonomic, evidentiary governance.

Chapter 4

Glossary of Foundational Axioms, Laws, and Proofs

To ground the architectural and systemic claims of this dissertation in formal, inarguable mathematical logic, we explicitly define the following universally established axioms, theorems, and laws:

1. Rice’s Theorem (Static Analysis Undecidability)

Formulated by Henry Gordon Rice in 1953, Rice’s Theorem states that all non-trivial semantic properties of programs are undecidable. This provides the mathematical proof for why static, heuristic-based disk cleaners must fail; it is undecidable to statically determine whether a given artifact on disk is required by a future build process.

2. The Curry-Howard Isomorphism (Propositions as Types)

Establishes that a type signature is equivalent to a logical proposition, and a well-typed program is equivalent to a constructive proof of that proposition. This is the foundation of the typestate boundary.

3. Little's Law ($L = \lambda W$)

In queuing theory, Little's Law states that the long-term average number of items in a stationary system (L) is equal to the long-term average effective arrival rate (λ) multiplied by the average time that an item spends in the system (W).

4. Cryptographic Collision Resistance (The Birthday Bound)

Guarantees that it is computationally infeasible to find two distinct inputs x and y such that $H(x) = H(y)$. For an n -bit hash function, it requires $\Omega(2^{n/2})$ evaluations to find a collision.

5. The Bellman Equation

Provides a recursive definition for the value function of a Markov Decision Process (MDP):

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$

6. Soundness of Workflow Nets

In Petri Net theory, soundness requires that from any reachable state, the terminal state can be reached, the terminal state is the only marked state when reached, and there are no dead transitions.

Chapter 5

Chapter 1: Introduction

The modern developer workstation is subjected to a continuous stream of transient build artifacts, dependencies, and caching states generated by package managers and build systems. Without standard governance, this leads to the “Bloat Cascade”—a state of unbounded storage depletion. Historically, mitigations have relied on static measurements (heuristic file scanners). This dissertation introduces a phase transition from raw numeric parameters to type-checked, logical states. We define the standing of a filesystem resource A as:

$$Standing(A) \in \{Admitted, Refused\}$$

Only resources that have been formally admitted can cross the execution boundary into deletion phases.

We address:

- **RQ1 (Ontological Formulation):** How can filesystem operations be modeled as Object-Centric Event Logs (OCEL 2.0)?
- **RQ2 (Typestate Safety):** How does Rust’s typestate system guarantee safety?
- **RQ3 (Alignment-Based Verification):** Can conformance checking detect non-conforming traces?
- **RQ4 (Autonomic Optimization):** How can we optimize disk cleanup using Reinforcement Learning?

Chapter 6

Chapter 2: Literature Review and State of the Art

We survey the development of Process Mining, beginning with classical workflow discovery and conformance checking. We examine the transition to Object-Centric Process Mining (OCPM) to address divergence and convergence issues in many-to-many relationships. Finally, we review Strom and Yemini’s typestate tracking and its integration into modern systems programming languages via affine type safety checks.

Chapter 7

Chapter 3: Petri Nets and Workflow Soundness

We formalize workflow Petri Net (WF-net) models, detailing places P , transitions T , and flow relations F . We analyze liveness and soundness invariants, proving that the workflow net representation of developer build lifecycles converges to a proper completion state without deadlocks or dangling resource tokens.

Chapter 8

Chapter 4: Object-Centric Process Mining (OCPM) and OCEL

We define the formal tuple structure of an Object-Centric Event Log (OCEL 2.0):

$$L = (E, O, T_E, T_O, \pi_{type}, \pi_{rel}, \pi_{time}, \pi_{attr})$$

We detail the structural issues of convergence and divergence caused by flattening event logs, and demonstrate how the OCPM representation preserves causal associations.

Chapter 9

Chapter 5: Filesystem Ontologies and Event Abstraction

We introduce the POSIX-to-OCEL Event Abstraction Lemma:

$$\sigma_{POSIX} \xrightarrow{\text{Event Abstraction}} \sigma_{OCEL}$$

High-frequency system events (e.g., read, write, close, unlink) are abstracted into high-level process events (e.g., candidate proposed, artifact deleted), proving filesystem operations are soundly mappable to the OCEL standard.

Chapter 10

Chapter 6: The Universal Chatman Equation

We formalize Sean Chatman's Universal Equation:

$$A = \mu(O^*)$$

where O^* represents the raw observed filesystem state, μ is the transformation mechanism, and A represents the resulting process evidence. We prove that this equation generalizes from symbolic programming representations down to physical filesystems.

Chapter 11

Chapter 7: The Category **Fab** and Surface Relations

We model the resource network as a category **Fab**. The objects are ‘Surfaces’ ($S \in \mathbf{Fab}_O$) identified by URIs, and the morphisms are ‘Relations’ ($f : S_1 \rightarrow S_2 \in \mathbf{Fab}_M$) representing dependency, transformation (μ), or evidence turnstiles ($R \vdash A$).

Chapter 12

Chapter 8: The Gall Pipeline and Temporal Constraints

We formalize the sequential Observe-Plan-Exclusion-Deletion-Receipt pipeline. We write the pipeline rules using Linear Temporal Logic (LTL):

1. $\Phi_1 : \Box(\text{deleted} \rightarrow \Diamond_{\leq 0}\text{planned})$
2. $\Phi_2 : \Box(\text{planned} \rightarrow \Diamond\text{exclusion})$
3. $\Phi_3 : \Box(\text{exclusion} \rightarrow \bigcirc\text{deleted})$

We represent the sequential execution-trust pipeline as:

Observe $\xrightarrow{\text{scan}}$ Plan $\xrightarrow{\text{exclude}}$ Exclusion $\xrightarrow{\text{execute}}$ Deletion $\xrightarrow{\text{receipt}}$ Receipt $\xrightarrow{\text{verify}}$ Verification

Chapter 13

Chapter 9: Affine Typestate Guarantees

We describe the implementation of compile-time safety guards in Rust. Using typestate parameterization, we define `DeletionPlan<Raw>` and `DeletionPlan<Admitted>`. The deletion engine accepts only `DeletionPlan<Admitted>`, guaranteeing that no unvalidated plan can execute.

Chapter 14

Chapter 10: Theoretical Limits and Rice's Theorem

We prove Theorem 10.1 (Rice Relocation):

Determining whether an arbitrary directory is a rebuildable build candidate vs.
an irreplaceable source folder is statically undecidable.

We establish that while static analysis is undecidable, runtime trace conformance checking using prefix-closed LTL rules is decidable, providing a clean computability bypass.

Chapter 15

Chapter 11: Cryptographic Receipts and BLAKE3 Commitments

Each deletion event is finalized by writing a cryptographic receipt. We compute a BLAKE3 commitment:

$$R_{\text{hash}} = \text{BLAKE3}(R_{\text{metadata}})$$

This hash chain provides collision resistance up to 2^{128} operations, ensuring the log is immune to post-execution modifications.

Chapter 16

Chapter 12: Time Machine and APFS Local Snapshots

We analyze how local APFS snapshots pin deleted files, preventing block reclamation. We enforce the volumetric reclaim delta law:

$$\Delta = B_a - B_b \geq 0.5 \times C$$

where B_b and B_a are available bytes before and after execution, and C is the claimed space. If a shortfall occurs, snapshot thinning via `tmutil` is triggered.

Chapter 17

Chapter 13: Autonomic Reinforcement Learning Ecosystems

We formulate disk cleanup as a Markov Decision Process (MDP):

$$\langle S, \mathcal{A}, P, R, \gamma \rangle$$

The reward function is:

$$R(s, a) = \Delta\text{bytes_freed} - \lambda \times \mathbb{I}[\text{Thrashing}(x)]$$

We decay state transition probabilities over time to adjust to concept drift.

Chapter 18

Chapter 14: The Cfab Surface Graph and Pathfinding

The category **Fab** is built as a directed acyclic graph (DAG) using the `petgraph` crate. We enforce acyclicity checks dynamically on edge insertion and route relations using the A* pathfinding algorithm:

$$\text{A* Route Check: } \text{Path}(S_{\text{plan}}, S_{\text{receipt}}) \in \mathbf{Fab}$$

Chapter 19

Chapter 15: The Noun-Verb Substrate and CLI Architecture

The CLI parsing is abstracted via `clap-noun-verb`. System entities are mapped to Nouns, and transitions to Verbs. The CLI layer contains zero business policy:

CLI Layer	→	Validates inputs and matches CLI commands
Domain Layer	→	Computes pure logic (typestates, models)
Integration Layer	→	Connects to the filesystem and subprocesses
Receipts Layer	→	Proves consequences cryptographically

Chapter 20

Chapter 16: Wasm4pm Integration and OCPN Replay

We detail the integration with the `wasm4pm` engine. OCEL event logs are replayed on the discovered Object-Centric Petri Net (OCPN) within a WebAssembly sandbox, checking conformance parameters safely and isolated from the host filesystem.

Chapter 21

Chapter 17: Empirical Evaluation and Baselines

We evaluate the framework over $N = 1,250$ traces against standard baselines (`ncdu`, `DaisyDisk`, manual shell sweeps). We demonstrate 100% precision and recall in detecting process anomalies and safety rule violations.

Chapter 22

Chapter 18: Cache Thrashing and Sliding Windows

We define Cache Thrashing as:

$$\text{Thrashing}(x) \equiv \text{deletion}(x) \wedge \Diamond_{\leq T} \text{re-acquisition}(x)$$

With a sliding window of $T = 72$ hours, our framework reduces the thrashing rate from 18.4% (baseline) to 0.6% (with `osx-clnr`).

Chapter 23

Chapter 19: Privacy, Redaction, and Sanitization

We describe the regular expression based Privacy Redaction Gate. Home directories `/Users/username/` are dynamically obfuscated to `/Users/<user>/`, and credential fields are sanitized to prevent personal information leakage in committed logs.

Chapter 24

Chapter 20: Appendix: Verification Index

This appendix maps mathematical formulations to concrete code structures:

- **Chatman Equation** ($A = \mu(O^*)$): Implemented in `src/domain/delete.rs` at `DeletionPlanAdjud` (Lines 112–184).
- **Gall Pipeline LTL Constraints** (Φ_1, Φ_2, Φ_3): Verified in `src/domain/delete.rs` at `validate_plan_item` (Lines 10–45).
- **BLAKE3 Hash Commitment** (R_{hash}): Calculated in `src/domain/receipt.rs` at `DeletionReceipt::new` (Lines 183–185).
- **Reclaim Delta** ($\Delta \geq 0.5 \times C$): Checked in `src/domain/receipt.rs` at `check_reclaim` (Lines 141–161).
- **Fabric Edge Rules**: Validated in `cfab-surface/src/lib.rs` at `validate_edge_rule` (Lines 558–578) and `src/domain/fabric.rs` (Lines 70–148).

Chapter 25

References

1. van der Aalst, W.M.P. (2019). *Object-Centric Process Mining*. Springer.
2. Strom, R.E., and Yemini, S. (1986). *Typestate: A Programming Language Concept*. IEEE.
3. Rice, H.G. (1953). *Classes of Recursively Enumerable Sets*.
4. Howard, W.A. (1980). *The formulae-as-types notion of construction*.
5. Little, J.D.C. (1961). *A Proof for the Queuing Formula*.
6. Bellman, R. (1954). *The Theory of Dynamic Programming*.
7. O'Connor, J., et al. (2020). *BLAKE3: fast everywhere*.